C-Lite Interpreter Project Report

Department of Computer Engineering
University of Peradeniya
CO523 - Programming Languages

---

1. Introduction

This project involved the design and implementation of an interpreter for **C-Lite**, a simplified imperative programming language that is a subset of C. The implementation covers lexical analysis, syntax analysis (parsing), and semantic evaluation (interpreting).

2. Grammar Specification (EBNF)

The following formal grammar defines the syntax of C-Lite:

```
program         ::= { statement }
statement       ::= declaration | assignment | if_statement | print_statement | compound_stat
declaration     ::= type identifier ";"
type            ::= "int" | "float"
assignment      ::= identifier "=" expression ";"
if_statement    ::= "if" "(" comparison ")" statement [ "else" statement ]
print_statement::= "printf" "(" identifier ")" ";"
compound_statement ::= "{" { statement } "}"
comparison      ::= expression ( ">" | "<" | "==" ) expression
expression      ::= term { ( "+" | "-" ) term }
term            ::= factor { ( "*" | "/" ) factor }
factor          ::= identifier | float_literal | int_literal | "(" expression ")"
```

3. Design Discussion

Architectural Phases

1. **Lexical Analysis (Scanner)**: The source code is scanned and converted into a stream of tokens (Keywords, Identifiers, Literals, Operators, Symbols).
2. **Syntax Analysis (Parser)**: A **Recursive Descent Parser** validates the token stream against the EBNF grammar and constructs an **Abstract Syntax Tree (AST)**.
3. **Semantic Evaluation (Interpreter)**: The AST is traversed using a visitor-like pattern. A **Symbol Table** manages variable declarations, bindings, and scoping.

Implementation Challenges

- **Operator Precedence**: Handled within the recursive descent parser by nesting production rules (Expression > Term > Factor).
- **Scoping Rules**: Implemented block scoping using a hierarchical Symbol Table. Each compound statement (`{ ... }`) creates a new scope linked to its parent.
- **Type Handling**: The interpreter supports basic `int` and `float` types. Variable declarations are required before assignment.

4. Test Suite and Verification

The interpreter was verified using sample programs covering various features.

Arithmetic and Floating Point

File: `arithmetic.clite`

```
int x;
int y;
x = 10;
y = 20;
int z;
z = x + y * 2;
printf(z); // Output: 50

float a;
a = 10.5;
float b;
b = a / 2.0;
printf(b); // Output: 5.25
```

Conditional Logic and Scoping

File: `conditionals.clite`

```
int age;
age = 18;

if (age == 18) {
    int status;
    status = 1;
    printf(status); // Output: 1
} else {
    int status;
    status = 0;
    printf(status);
}
```

```
if (age > 20) {
    int x;
    x = 100;
    printf(x);
} else {
    int y;
    y = 200;
    printf(y); // Output: 200
}
```

5. Conclusion

The C-Lite interpreter successfully demonstrates the translation process from source code to execution. The modular design ensures that each phase (Scanning, Parsing, Interpreting) is clearly separated and maintainable.